Revocable Academic Credentials for Kazakhstan

A controlled comparison of individual and Merkle-batched registration

CSCI 435 Research Track | Stage 3 proposal | Prepared 10 October 2026

Team: Nurbek Bauyrzhan and Aldiyar Sagat.
Status: Working on the project.

1. Problem and scope

We propose a small Ethereum-compatible registry through which an authorized university issues a commitment to an academic document, a graduate retains the document, and an employer checks integrity and current status. The Kazakhstan setting motivates the issuer-governance and deployment discussion; experiments will use synthetic records, not real NU diplomas, student identifiers, grades or state-system access. The product is a command-line prototype and reproducible evaluation, not a national platform or a university web portal.

Working gap (one sentence). For our credential-verification setting, the unresolved design decision is when Merkle batching reduces lifecycle gas relative to individual records under the same issuer and credential-revocation policy, once verification work and proof availability are accounted for.

This is a *candidate empirical gap*, not a claim that batching or blockchain credentials are new. Blockcerts already supplies blockchain credential tools [1], and Kazakhstan-specific transcript and educational-document systems have been published [2,3]. The latter also discusses batch commitments. Our proposed contribution is a transparent workload comparison, including cases where savings disappear. Before claiming novelty, we will compare the exact workload, status semantics and metrics against the team's bibliography and the required Moodle readings; if already covered, we will frame the study as a reproduction with an explicitly justified extension and seek instructor approval.

2. Research questions and comparison

Primary question: How do batch size and revocation workload affect the gas trade-off between individual and Merkle-batched academic credential registration under a common trust model?

1. How does issuance gas per credential change with batch size?
2. What proof-size and verification-work overhead does batching introduce?
3. How do individual revocations and issuer suspension/restoration affect cost and correctness?

Baseline A: One on-chain record and issuance transaction per credential.
Alternative B: One on-chain Merkle root per batch, with off-chain membership proofs and on-chain per-credential revocation. This is an engineering baseline comparison, not a replication of the gas numbers in [2,3].

3. System and threat model

An administrator authorizes issuer addresses. An authorized university issues and revokes credentials; a verifier uses a trusted chain and registry address. A record binds an identifier to a salted document commitment. Document bytes and a random 32-byte salt remain off-chain; the registry exposes issuer and status information. The batch leaf additionally binds issuer, batch identifier and credential identifier. We will use existing SHA-256/Keccak libraries, not implement cryptographic primitives.

An attacker may modify documents, submit unauthorized transactions, substitute proofs or present revoked credentials. Tests must reject these cases. We assume uncompromised issuer/admin keys and correct chain execution; a dishonest authorized university can issue false claims. A matching commitment proves integrity and registry status, not identity, accreditation or educational achievement. Salt loss or unavailable proofs can prevent verification. Salting does not eliminate metadata leakage.

For a fair comparison, each experimental credential appears once. Revocation is irreversible; issuer suspension is reversible but cannot undo explicit revocation. Duplicate credentials across batches will be tested separately because batch-scoped revocation is not global cancellation. Corrections require explicit revocation and reissuance. Whole-batch revocation is a separate operation, not an equivalent baseline workload.

4. Evaluation and reproducibility

Environment and data. Solidity 0.8.24, optimizer 200 runs, Shanghai compiler target; Python 3.12 and a local EVM. Record the actual backend fork and all dependency versions. Generate synthetic byte strings and salts with fixed experiment seed 42; production-style packaging uses cryptographic randomness. Use fresh deployments for each configuration and five repeats. Initial pilot sizes are 1, 8, 32 and 128; extend to 3, 7, 31, 127 and 512 to test odd-tree handling and scale.

Workload. For each size n, use identical credentials in both methods. Issue all records, then revoke $k = \lfloor rn \rfloor$ distinct records for $r \in \{0, 0.01, 0.10, 0.50\}$. Report actual k, including zeros caused by rounding. Record deployment and authorization separately. Validate every generated credential before and after revocation; test altered documents, unauthorized issuers, invalid proofs and suspension/restoration. No fraud-classification precision or recall is claimed.

Metrics. Measure issuance/revocation gas from mined receipts, proof bytes as 32 times the sibling count, and Merkle construction time in milliseconds. Measure verification gas with a consistently specified transaction simulation and label estimates as estimates. Ordinary read-only `eth_call` verification does not incur a user transaction fee. If reporting local call duration, use three warm-ups and twenty measured calls per configuration; report repeat-level medians and variation, not network latency.

Define measured write cost per credential as

$$C_{\text{write}}(n,k) = \frac{G_{\text{issue}}(n) + \sum_{j=1}^{k} G_{\text{revoke},j}}{n}.$$

Report setup costs separately. For a hypothetical paid on-chain verifier, also evaluate $C_{\text{write}} + q\widehat{G}_{\text{verify}}$ for $q \in \{0, 1, 10\}$ checks per credential, clearly separating estimated verification gas from receipt measurements. This sensitivity analysis does not represent the cost of ordinary off-chain verification.

Captions fixed before the new experiments.

- **Table 1:** Individual versus batch registration by n and actual k: issuance gas/credential, revocation gas, verification gas estimate and proof bytes; five-repeat mean and standard deviation.
- **Figure 1:** Write gas per credential (gas units, vertical axis) against batch size (credential count, horizontal axis); method and revocation-fraction curves, with repeat variation.
- **Figure 2:** Estimated total gas per credential against paid checks per credential q; compare methods at $n = 128$ for each revocation fraction. Caption distinguishes hypothetical paid checks from free read-only calls.

Raw CSV files and metadata belong in `data/`; scripts in `src/` generate vector PDFs in `figures/`. Record seeds, compiler/backend settings, source revision and excluded costs. No external data scraping is required.

5. Delivery, team roles and limitations

Suggested division: Nurbek owns the Python generator, benchmark and figures; Aldiyar owns contracts and lifecycle tests. Both review each other's code, verify literature and co-write the paper.

Schedule: Week 9: approve scope, build individual baseline and tests. Week 10: add batching and reproduce a pilot. Week 11: run the workload grid and draft method/evaluation. Week 12: independent clean-clone reproduction. Week 13: full 10–15-page LaTeX draft, cross-team review and response. Week 14: finalize repository/paper and submit after instructor clearance. Each member makes substantive current-dated commits through their own account and attends weekly checks.

Week 10 fallback: If batching is unreliable, report the working individual registry, measured costs and documented failure cases; narrow the question with the instructor. If testnet access fails, retain the local EVM study. Synthetic data and one EVM configuration limit generalization. Deployment discussion will examine issuer onboarding, key recovery, credential delivery, existing university/state workflows and current Kazakhstan rules; no NU/eGov integration, legal compliance or superiority over signed databases is claimed.

Selected references (not the full annotated bibliography)

[1] Blockcerts, "Introduction," project documentation, https://www.blockcerts.org/guide/, accessed 10 Oct. 2026.

[2] O. Ussatova et al., "Application of Blockchain Technologies and Smart Contracts for the Storage and Verification of Academic Transcripts in the Higher Education Systems," *Information*, 17(5), 478, 2026. https://doi.org/10.3390/info17050478.

[3] O. Ussatova et al., "The Blockchain-Based Multifunctional Educational Documents Verification System," *Information*, 17(8), 720, 2026. https://doi.org/10.3390/info17080720.